Discrete & Continuous Models & Applied Computational Science
ISSN 2658-7149 (Online), 2658-4670 (Print)

2026, 230 (1) 1-??
http://journals.rudn.ru/miph

Research article

UDC 004.729, 519.872
PACS 89.20.Ff, 02.60.Pn, 07.05.Tp
DOI: 10.22363/2658-4670-2026-230-1-1-**??**
EDN: XXXXXX

Comparative Analysis of Active Queue Management Algorithms: RED, ARED, Gentle RED

Abuli M. Palagashvili[1]

[1] *RUDN University, 6 Miklukho-Maklaya St, Moscow, 117198, Russian Federation*

Abstract. This paper presents a systematic comparative analysis of three Active Queue Management (AQM) algorithms from the RED family: RED (Random Early Detection), ARED (Adaptive RED), and Gentle RED. A dumbbell-topology network model is implemented in the ns-3 simulator with ten TCP bulk-send sources creating sustained congestion at a bottleneck link. All three algorithms are evaluated under identical conditions (same queue thresholds, load profile, and topology). An extended set of metrics is proposed, including EWMA zone distribution and temporal drop structure. Results show that while all algorithms achieve the same average throughput (~90% link utilization), they exhibit fundamentally different buffer management dynamics: RED provides the most predictable throughput (CV = 18.7%) at the cost of highest drop count; ARED minimizes drops (−19% vs. RED) but produces the largest queue oscillations (p99 = 49 packets); Gentle RED achieves the lowest average queue length (6.40 packets) with the highest throughput variability (CV = 24.7%). Practical recommendations for algorithm selection based on scenario priorities are formulated.

Key words and phrases: active queue management, AQM, RED, Adaptive RED, Gentle RED, congestion control, ns-3, network simulation, bufferbloat

1. Introduction

The continuous growth of Internet traffic, driven by the development of cloud computing, video streaming, and 5G/6G networks, imposes increasingly stringent requirements on congestion management mechanisms in network nodes. The traditional approach—tail-drop, in which packets are discarded only when the buffer is completely full—leads to global synchronization of TCP flows, sharp oscillations of throughput, and the bufferbloat effect [1]. The bufferbloat problem—systematic buffer overflow in routers causing end-to-end delay growth by orders of magnitude—has been identified as one of the key threats to quality of service [2].

Active Queue Management (AQM) algorithms, recommended by IETF in RFC 2309 [3], solve this problem by preemptive dropping or marking of packets before buffer overflow. The RED (Random Early Detection) family of algorithms remains the most widely deployed class of AQM mechanisms: RED is implemented in all major network operating systems (Cisco IOS, Linux tc, FreeBSD ALTQ), and its variants—ARED and Gentle RED—are shipped as standard modules in the ns-3 simulator and the Linux kernel. Despite the emergence of new algorithms (CoDel [4], PIE [5]), the choice of a specific RED variant for specific operating conditions still requires experimental justification, since analytical models do not account for all aspects of interaction with heterogeneous TCP traffic.

1.1. Literature review

The RED algorithm was proposed by Floyd and Jacobson in 1993 [6] and became the first practical AQM mechanism. The authors demonstrated that probabilistic early dropping of packets based on the exponentially weighted moving average (EWMA) of queue length avoids global synchronization of TCP flows and reduces average buffer delay compared to tail-drop.

Practical deployment of RED revealed its main drawback—high sensitivity to parameter tuning. Christiansen et al. [7] showed that RED performance significantly depends on the ratio of $\min_{\text{Th}}$, $\max_{\text{Th}}$, and maxP, with optimal values changing as the number of flows and load level change. To overcome this limitation, Floyd et al. [8] proposed ARED (Adaptive RED), in which the maxP parameter automatically adapts to the current load based on the deviation of the average queue from the target range.

In parallel, Floyd [9] proposed the Gentle RED modification, which eliminates the sharp transition of drop probability from maxP to 1 at $\bar{q} = \max_{\text{Th}}$. Gentle RED introduces an additional linear zone $[\max_{\text{Th}}, 2 \cdot \max_{\text{Th}})$, in which the probability smoothly increases to unity, reducing TCP flow synchronization and decreasing delay variance during short-term bursts.

A significant contribution to mathematical modeling of TCP and RED interaction was made by Korolkova A.V. [10], who constructed a mathematical model of the traffic transmission process with dynamic flow intensity regulated by a RED-type algorithm. In her work, using the apparatus of stochastic differential equations with a Poisson process, an autonomous system of three ordinary differential equations was formalized for the first time, describing the joint dynamics of TCP window size, instantaneous queue length, and EWMA average. A method was proposed for determining the region of self-oscillation occurrence, and qualitative analysis of the model was performed using RED, GRED, DSRED, and ARED algorithms in the NS-2 simulator.

The development of this direction was continued by Velieva T.R. [11], who investigated nonlinear phenomena and boundaries of the self-oscillation region in continuous-discrete dynamical models. The key contribution of this work was the adaptation of the Krylov–Bogolyubov method and harmonic linearization method to the continuous-discrete TCP/RED model, which made it possible to analytically determine the amplitude-frequency characteristics of self-oscillations and the boundaries of their occurrence region in the parameter space.

Among alternative approaches to AQM, the BLUE algorithm [12], which controls drop probability based on loss events and link idle events, and the family of control-theory-based algorithms—the PI controller of Hollot et al. [13] and its development PIE [5]—should be noted. The CoDel algorithm [4] uses a fundamentally different approach, tracking the sojourn time of a packet in the queue instead of queue length.

Thus, despite extensive research on individual aspects of the RED family algorithms—both analytical (stability, self-oscillations) and experimental—a systematic three-way comparison of RED, ARED, and Gentle RED under strictly identical conditions with an extended set of metrics (including temporal drop structure and zone distribution of the queue) is insufficiently represented in the literature.

1.2. Research objectives

The **goal** of this work is to conduct a comparative analysis of three algorithms from the RED family (RED, ARED, Gentle RED) under conditions of sustained congestion of a network node and to identify quantitative differences in buffer management dynamics, throughput stability, and the nature of congestion signals.

To achieve this goal, the following **tasks** are set:

1. Implement a simulation model of a network with dumbbell topology in the ns-3 environment with the ability to switch the AQM algorithm while maintaining other conditions.

2. Conduct a series of experiments with identical queue parameters and load for each of the three algorithms.
3. Compare algorithms by a set of metrics: mean and tail queue length, coefficient of variation of throughput, number and temporal structure of packet drops.
4. Formulate practical recommendations for AQM algorithm selection depending on the priority requirements of the scenario.

1.3. Scientific novelty

The scientific novelty of this work consists of the following:

1. Unlike the works of Floyd and Jacobson [6], Floyd [8, 9], and Christiansen et al. [7], where each algorithm was analyzed in isolation or in pairwise comparison with tail-drop, this work conducts a systematic three-way comparison of RED, ARED, and Gentle RED under strictly identical conditions (unified topology, identical thresholds, identical load profile), which eliminates the influence of methodological differences on results.
2. Unlike the analytical approach of Korolkova [10], who studied self-oscillation modes using ODE systems, and Velieva [11], who applied the harmonic linearization method to determine stability boundaries, this work proposes an extended set of *experimental* evaluation metrics, including zone distribution of EWMA average operating time and temporal structure of drops (median interval, clustering), which allows characterizing not only static indicators but also the dynamics of control actions in the simulation model.
3. Unlike the works of Hollot et al. [13] and Feng et al. [12], focused on comparing RED with fundamentally different AQM approaches (PI controller, BLUE), this work focuses on *intra-family* comparison of RED variants and for the first time quantitatively describes the fundamental trade-off between throughput predictability (RED, CV = 18.7%), drop minimization (ARED, −19%), and buffer delay minimization (Gentle RED, average queue 6.40 pkt).

1.4. RED family algorithms

Active queue management algorithms begin dropping or marking packets *preemptively*—before the buffer overflows—based on average queue length statistics. TCP connections receive congestion signals earlier and reduce their sending rate more smoothly, preventing collapse.

1.4.1. RED (Random Early Detection)

RED is the base AQM algorithm proposed by Floyd and Jacobson in 1993 [6]. The algorithm computes the exponentially weighted moving average of queue length (EWMA) and makes drop decisions based on it.

Average queue update on each arriving packet:

$$\bar{q} = (1 - w_q)\,\bar{q} + w_q\,q$$

where q is the instantaneous queue length, $w_q \in (0, 1)$ is the filter weight.

Drop probability is determined by three zones:

$$p(\bar{q}) = \begin{cases} 0 & \bar{q} < \min_{\text{Th}} \\ \text{maxP} \cdot \dfrac{\bar{q} - \min_{\text{Th}}}{\max_{\text{Th}} - \min_{\text{Th}}} & \min_{\text{Th}} \le \bar{q} < \max_{\text{Th}} \\ 1 & \bar{q} \ge \max_{\text{Th}} \end{cases}$$

The hard forced drop above $\max_{\text{Th}}$ ensures fast reaction but synchronizes TCP flows and creates sawtooth queue oscillations.

1.4.2. ARED (Adaptive RED)

ARED extends RED with an adaptive mechanism [8]: the maxP parameter is periodically recomputed every T_{interval} depending on the deviation of the average queue from the target range:

$$\text{maxP} = \begin{cases} \text{maxP} + \alpha & \bar{q} > \dfrac{\min_{\text{Th}} + \max_{\text{Th}}}{2} \\ \text{maxP} - \beta & \bar{q} < \dfrac{\min_{\text{Th}} + \max_{\text{Th}}}{2} \end{cases}$$

where α and β are increment and decrement coefficients ($\alpha > \beta$). The value of maxP is bounded within an admissible range. This allows the algorithm to adapt to changing load without manual tuning; however, the adaptation introduces additional inertia and may lead to wide queue oscillations.

1.4.3. Gentle RED

Gentle RED [9] modifies RED behavior in the zone above $\max_{\text{Th}}$: instead of immediate forced drop, the algorithm continues the probabilistic approach up to $2 \cdot \max_{\text{Th}}$:

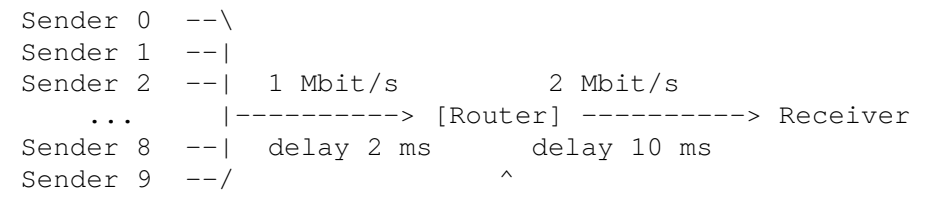
$$p(\bar{q}) = \begin{cases} 0 & \bar{q} < \min_{\text{Th}} \\ \text{maxP} \cdot \dfrac{\bar{q} - \min_{\text{Th}}}{\max_{\text{Th}} - \min_{\text{Th}}} & \min_{\text{Th}} \le \bar{q} < \max_{\text{Th}} \\ \text{maxP} + (1 - \text{maxP}) \cdot \dfrac{\bar{q} - \max_{\text{Th}}}{\max_{\text{Th}}} & \max_{\text{Th}} \le \bar{q} < 2\max_{\text{Th}} \\ 1 & \bar{q} \ge 2\max_{\text{Th}} \end{cases}$$

In the zone $[\max_{\text{Th}}, 2 \cdot \max_{\text{Th}})$ the probability linearly grows from maxP to 1, and forced drop occurs only at $\bar{q} \ge 2 \cdot \max_{\text{Th}}$. This softens the reaction to short-term bursts, reduces TCP flow synchronization, and decreases the average queue length—at the cost of somewhat greater throughput variability.

2. Methods

2.1. Experimental topology

The experiment uses a dumbbell topology: multiple senders are connected to a common router through fast links, and the router is connected to a single receiver through a narrow bottleneck link with limited bandwidth. The narrow link is the only possible congestion point, and the AQM queue discipline is installed on its outgoing interface.



```
AQM queue
(RED / ARED / Gentle)
```

Each of the ten senders establishes one TCP connection with the receiver and continuously transmits data (BulkSend). The total incoming load on the router potentially reaches 10×1 Mbit/s $= 10$ Mbit/s, which is 5 times the bottleneck link capacity. This ensures sustained congestion and allows evaluation of AQM algorithm behavior under real buffer saturation conditions.

Topology parameters are listed in Table 1.

Table 1

Topology and simulation parameters

Parameter	Value	Description
`nSenders`	10	Number of TCP senders
`fastRate`	1 Mbit/s	Sender → router link speed (delay 2 ms)
`bottleneckRate`	2 Mbit/s	Bottleneck link speed (router → receiver)
`bottleneckDelay`	10 ms	Bottleneck link delay
`simTime`	60.0	Simulation duration (s)
Transport	TCP BulkSend	Segment size 512 bytes

2.2. AQM configuration

All three algorithms are launched with identical threshold parameters (Table 2).

Table 2

Unified threshold parameters for RED, ARED, and Gentle RED

Parameter	Value	Description
`minTh`	5 packets	Lower average queue threshold—no drops below
`maxTh`	10 packets	Upper threshold; above it RED/ARED drop all packets
`MaxSize`	100 packets	Hard buffer limit

2.3. Instrumentation

The discrete-event network simulator **ns-3** version 3.41 was used. Simulation scripts are placed in the `scratch/` directory. For this experiment, a subdirectory was created with the following structure:

```
ns-3.41/
+-- scratch/
    +-- red-experiments/
        +-- red.cc
        +-- configs/
```

```
|   +-- red.txt
|   +-- ared.txt
|   +-- gentle.txt
+-- analysis/
    +-- vis.py
    +-- ...
```

Configuration files specify the AQM type and queue parameters. Example `configs/red.txt`:

```
aqmType=red
minTh=5
maxTh=10
MaxSize=100
```

Analogous files for ARED (`aqmType=ared`) and Gentle RED (`aqmType=gentle`) differ only in the `aqmType` field value.

Simulations were launched with:

```
./ns3 run "scratch/red-experiments/red.cc --config=configs/<algorithm>.tx
```

Upon completion of each simulation, three CSV files were generated:

- `<algorithm>_queue.csv`—queue length sampled every 100 ms;
- `<algorithm>_drops.csv`—timestamps of each packet drop;
- `<algorithm>_throughput.csv`—throughput sampled every 100 ms.

For plotting, the script `vis.py` with moving average smoothing (window of 30 samples) was used:

```
python vis.py --algorithms red ared gentle --window 30
```

2.4. Evaluation metrics

Table 3

Algorithm evaluation metrics

Metric	Data source	What it characterizes
Mean and median queue length	`*_queue.csv`	Average buffer delay
p90 / p99 queue length	`*_queue.csv`	Tail delays
Fraction of time with empty queue	`*_queue.csv`	Link underutilization
Mean throughput	`*_throughput.csv`	Link utilization
Coefficient of variation (CV)	`*_throughput.csv`	Flow stability
Total drop count	`*_drops.csv`	Load on TCP congestion mechanism
Median inter-drop interval	`*_drops.csv`	Nature of control signals

3. Results

The experiment consisted of three sequential simulations—one for each algorithm. Each algorithm was launched with identical queue parameters (Table 2) for 60 seconds (the first second is warm-up, excluded from analysis).

3.1. Queue dynamics

Figure 1 shows the queue length dynamics for the three algorithms over the entire simulation time.

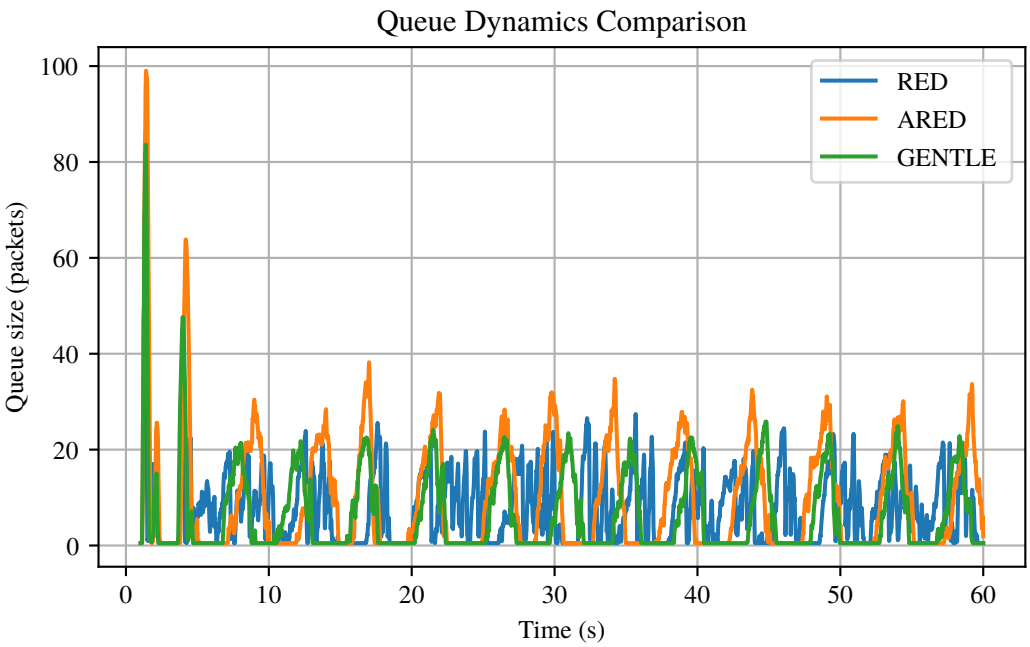


Figure 1. Queue length dynamics for RED, ARED, and Gentle RED

Two phases are clearly distinguishable on the plot.

Slow-start phase (t = 0–4 s). All TCP connections simultaneously enter slow-start and saturate the buffer within seconds. The queue for all algorithms reaches peak values: ARED—up to 100 packets, RED and Gentle RED—up to 84 packets. This burst is identical for all three, since the AQM algorithm has not yet accumulated sufficient statistics to react.

Steady-state regime (t > 5 s). After the initial burst dissipates, the algorithms diverge in behavior.

RED (blue curve) maintains the queue in the range of 5–26 packets with relatively regular oscillations. The curve rarely touches zero (empty queue only 13.4% of time): the forced drop at `m_qAvg >= maxTh` reacts quickly and keeps the queue "in check," but at the cost of constant pressure on TCP flows.

ARED (orange curve) demonstrates the largest oscillation amplitude and the most chaotic pattern. Periodically the queue grows to 30–35 packets in steady state and sharply collapses to zero. This is a consequence of maxP adaptation: the algorithm alternately "releases" and "tightens" the drop probability, producing wide irregular cycles. p99 = 49 packets—the highest value among the three algorithms.

Gentle RED (green curve) returns to zero most frequently (26.6% of time the queue is empty, median = 1 packet). The smooth probabilistic function in the zone `[maxTh, 2*maxTh]` allows TCP flows to desynchronize their windows, leading to deeper but shorter queue "dips."

Steady-state queue statistics are presented in Table 4.

Table 4

Queue length statistics in steady-state regime

Algorithm	Mean (pkt)	Std. dev.	Median	p90	p99	Empty (%)
RED	7.74	7.45	6	18	26	13.4%
ARED	9.57	11.94	4	25	49	22.3%
Gentle	6.40	8.86	1	19	34	26.6%

The time distribution across EWMA average $\bar{q}$ operating zones is shown in Table 5.

Table 5

Fraction of time in each queue management zone

Algorithm	$\bar{q} < \min_{\text{Th}}$	$[\min_{\text{Th}}, \max_{\text{Th}})$	$[\max_{\text{Th}}, 2\max_{\text{Th}})$	$\bar{q} \geq 2\max_{\text{Th}}$
RED	46.0%	19.8%	29.3%	4.8%
ARED	52.6%	8.0%	19.6%	19.7%
Gentle	60.8%	11.8%	19.8%	7.7%

RED spends 34.1% of time in the forced drop zone (`m_qAvg >= maxTh`), explaining its high average queue. ARED spends 19.7% of time in the $\bar{q} \geq 2 \cdot \max_{\text{Th}}$ zone where all packets are dropped—hence its anomalously high p99. Gentle RED most frequently stays below minTh (60.8%) and least frequently enters the hard drop zone (7.7%).

3.2. Throughput

The throughput plot (Fig. 2) is constructed with moving average smoothing (window of 30 samples), thus displaying data starting from $t \approx 29$ s. The Y-axis is intentionally compressed to the range 1.74–1.86 Mbit/s, allowing examination of differences invisible on the full scale.

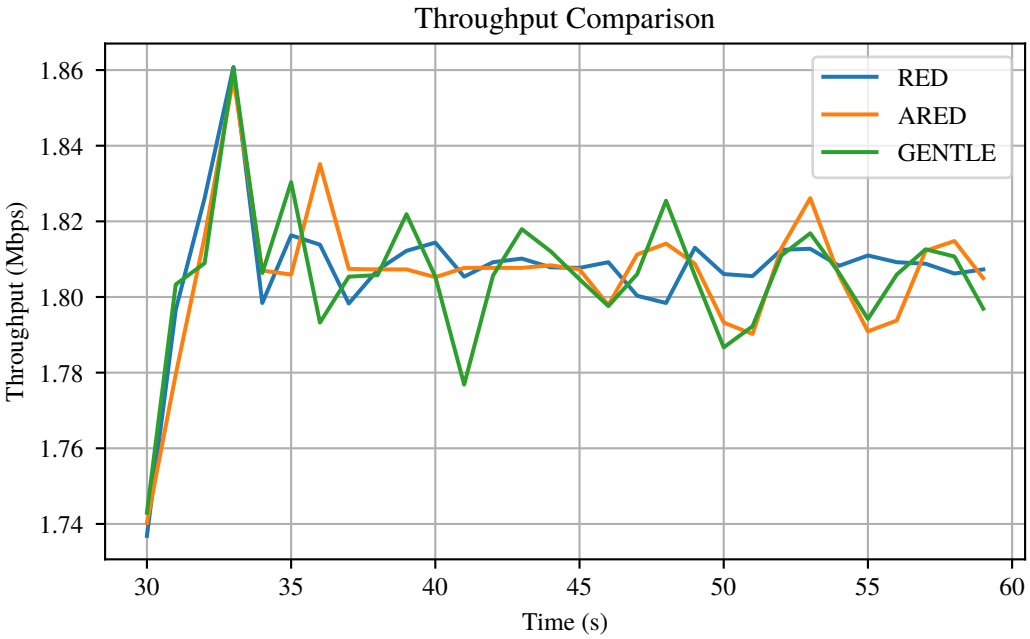


Figure 2. Throughput comparison: RED, ARED, and Gentle RED

All three algorithms stably maintain throughput near **1.80–1.81 Mbit/s** (~90% of the nominal 2 Mbit/s link capacity). This means that the choice of AQM algorithm at these parameters does not affect bandwidth utilization: all three mechanisms equally effectively "fill" the link.

The characteristic peak around $t = 33$ s (all curves reach ~1.86 Mbit/s) represents synchronous recovery of TCP flows after the initial wave of drops: congestion windows simultaneously begin growing, creating a short-term excess flow.

Table 6

Throughput statistics (CV—coefficient of variation)

Algorithm	Mean (Mbit/s)	Std. dev.	Min.	Max.	CV
RED	1.8047	0.3380	0.533	3.621	18.7%
ARED	1.8039	0.3950	0.553	3.293	21.9%
Gentle	1.7998	0.4444	0.352	3.654	24.7%

Despite identical means, **RED demonstrates the lowest variability** (CV = 18.7%): hard synchronous drops keep all TCP flows in step, making throughput predictable. Gentle RED has the highest CV (24.7%)—asynchronous window management creates more "ripple," although the mean does not suffer.

3.3. Packet drops

The plot (Fig. 3) presents drop moments as a scatter diagram: each point is one dropped packet. Three horizontal bands correspond to the three algorithms.

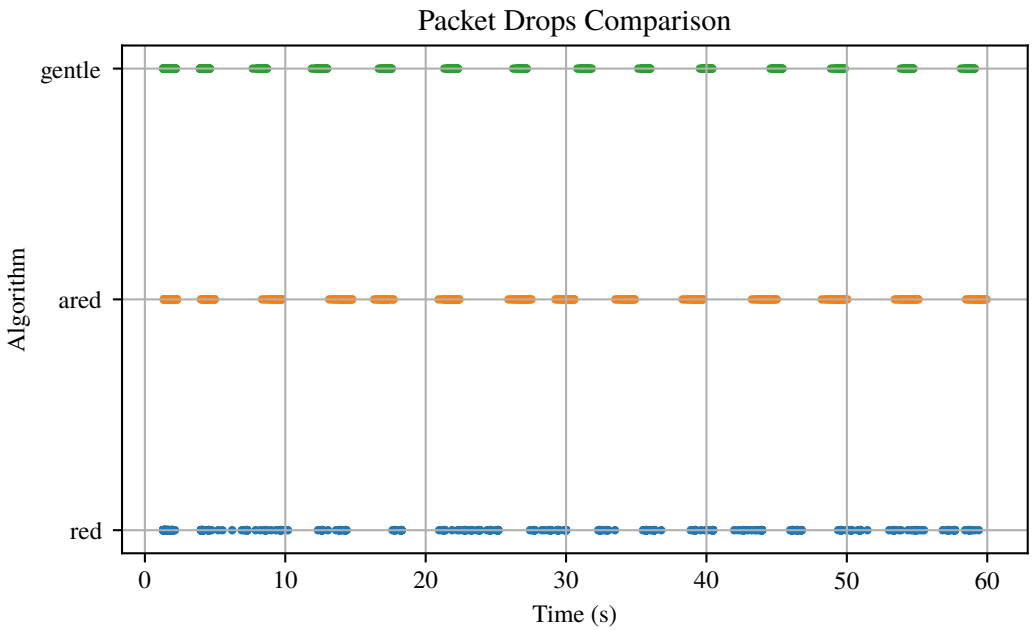


Figure 3. Packet drop moments for each algorithm

RED (bottom band, blue)—the densest and most continuous band: points fill virtually the entire simulation time, with only rare small gaps. RED actively drops in 47 out of 58 seconds with a median inter-drop interval of only **2.3 ms**. The forced drop at `m_qAvg >= maxTh` creates a continuous stream of dropped packets each time the average queue exceeds the threshold.

ARED (middle band, orange)—clearly visible drop clusters separated by noticeable pauses. The algorithm is active in 33 out of 58 seconds. The maxP adaptation periodically reduces drop probability to zero, giving flows a "break." However, when drops occur, their density within a cluster is high (maximum burst—276 drops per second).

Gentle RED (top band, green)—the most pronounced cluster structure with the longest pauses between clusters. The algorithm is active in only 27 out of 58 seconds. Within each cluster, drops are dense (maximum burst—305/s), but between them—extended "quiet" intervals.

Drop statistics summary is presented in Table 7.

Table 7

Packet drop statistics

Algorithm	Total drops	Rate (drops/s)	Median interval	Active seconds	Max. burst
RED	1240	21.34	2.3 ms	47	321
ARED	1003	17.11	11.3 ms	33	276
Gentle	1026	17.75	9.1 ms	27	305

RED generates **19–21% more drops** than the competitors. The median inter-drop interval for RED (2.3 ms) is 4–5 times smaller than for ARED and Gentle—RED practically continuously drops packets during congestion periods, while ARED and Gentle operate in "pulses."

4. Discussion

The final comparison of algorithms across all key metrics is presented in Table 8.

Table 8

Summary comparison of RED, ARED, and Gentle RED algorithms

Metric	RED	ARED	Gentle RED
Mean throughput	1.8047 Mbit/s	1.8039 Mbit/s	1.7998 Mbit/s
Throughput CV	**18.7%**	21.9%	24.7%
Mean queue	7.74 pkt	9.57 pkt	**6.40 pkt**
Queue std. dev.	7.45	11.94	8.86
p99 queue	26 pkt	49 pkt	34 pkt
Empty queue (%)	13.4%	22.3%	**26.6%**
Total drops	1240	**1003**	1026
Drop pattern	continuous	clustered	pulsed
Median inter-drop interval	2.3 ms	11.3 ms	9.1 ms

RED provides the most predictable and stable throughput (CV = 18.7%) but achieves this at the cost of the highest number of drops (1240) and the highest average queue. The continuous nature of drops synchronizes TCP flows, creating rigid, "quantized" congestion management dynamics.

ARED minimizes the number of drops (1003, −19% vs. RED) and least frequently disturbs flows. However, maxP adaptation creates significant queue instability (std. = 11.94, p99 = 49 packets) with periods of excessive buffer growth—potentially undesirable from a delay perspective.

Gentle RED achieves the lowest average queue (6.40 packets) and the highest fraction of empty buffer (26.6%). The number of drops is comparable to ARED (1026), but they are distributed in the most compact pulses with long pauses between them. The cost is the highest throughput variability (CV = 24.7%) due to TCP flow desynchronization.

5. Conclusion

The conducted experiment showed that all three AQM algorithms—RED, ARED, and Gentle RED—provide practically the same average throughput (~1.80 Mbit/s, ~90% of link capacity), but fundamentally differ in the nature of buffer management and the load on the TCP congestion control mechanism.

The key difference lies in the trade-off between predictability and aggressiveness of queue management. RED acts most aggressively: forced drop upon exceeding $\max_{\text{Th}}$ keeps the queue "in check" (mean 7.74 packets) and ensures minimum throughput variability (CV = 18.7%), but at the cost of the highest number of drops (1240) and continuous pressure on TCP flows. ARED, conversely, sacrifices queue stability for reduced drops: maxP adaptation decreases drops by 19% compared to RED, but produces wide oscillations (std. = 11.94, p99 = 49 packets), unacceptable for delay-sensitive applications. Gentle RED occupies an intermediate position, achieving the lowest average queue (6.40 packets) due to a smoother drop function that desynchronizes TCP flows and reduces buffer occupancy—but increasing throughput variability (CV = 24.7%).

The obtained results confirm that AQM algorithm selection is determined by specific scenario requirements: there is no single "best" solution—each algorithm is optimal for its target metric.

Application recommendations:

- When **minimum buffer delay** is priority—Gentle RED (lowest average queue 6.40 pkt).
- When **minimum drop count** is priority—ARED (19% fewer drops than RED).
- When **predictable throughput** is priority—RED (lowest CV 18.7%).

Bibliography

1. Gettys, J. & Nichols, K. Bufferbloat: Dark Buffers in the Internet. *Communications of the ACM* **55,** 57–65. doi:10.1145/2063176.2063196 (2012).
2. Nichols, K., Jacobson, V., McGregor, A. & Iyengar, J. *Controlled Delay Active Queue Management* RFC 8289. 2018.
3. Braden, B. *et al. Recommendations on Queue Management and Congestion Avoidance in the Internet* RFC 2309. 1998.
4. Nichols, K. & Jacobson, V. Controlling Queue Delay. *ACM Queue* **10,** 20–34. doi:10 . 1145 / 2209249.2209264 (2012).
5. Pan, R., Natarajan, P., Piglione, C., Prabhu, M. S., Subber, V., Khan, F. & Athiappan, K. *PIE: A Lightweight Control Scheme to Address the Bufferbloat Problem* in *Proceedings of IEEE International Conference on High Performance Switching and Routing (HPSR)* (2013), 148–155. doi:10.1109/HPSR.2013.6602305.
6. Floyd, S. & Jacobson, V. Random Early Detection Gateways for Congestion Avoidance. *IEEE/ACM Transactions on Networking* **1,** 397–413. doi:10.1109/90.251892 (1993).
7. Christiansen, M., Jeffay, K., Ott, D. & Smith, F. D. Tuning RED for Web Traffic. *IEEE/ACM Transactions on Networking* **9,** 249–264. doi:10.1109/90.929849 (2001).
8. Floyd, S., Gummadi, R. & Shenker, S. *Adaptive RED: An Algorithm for Increasing the Robustness of RED's Active Queue Management* tech. rep. (ICSI, 2001).
9. Floyd, S. *Recommendation on using the "gentle" variant of RED* https://www.icir.org/floyd/red/gentle.html.
10. Korolkova, A. V. *Mathematical Model of Traffic Transmission with Dynamic Flow Intensity Regulated by a RED-type Algorithm: PhD Thesis* (RUDN University, Moscow, 2010).
11. Velieva, T. R. *Analysis of Self-Oscillation Modes in Continuous-Discrete Dynamical Models: PhD Thesis* (RUDN University, Moscow, 2019).
12. Feng, W.-c., Kandlur, D. D., Saha, D. & Shin, K. G. *BLUE: A New Class of Active Queue Management Algorithms* tech. rep. CSE-TR-387-99 (University of Michigan, 2002).
13. Hollot, C. V., Misra, V., Towsley, D. & Gong, W.-B. *On Designing Improved Controllers for AQM Routers Supporting TCP Flows* in *Proceedings of IEEE INFOCOM* (2001), 1726–1734. doi:10.1109/INFCOM.2001.916670.